

BASI DI DATI:

Introduzione:

Ogni organizzazione è dotata di un sistema informativo, che organizza e gestisce le informazioni necessarie per perseguire gli scopi dell'organizzazione stessa. L'esistenza del sistema informativo è in parte indipendente dalla sua automatizzazione. Le informazioni vengono rappresentate per mezzo di dati, che hanno bisogno di essere interpretati per fornire informazioni.

Base di dati:

Una base di dati è un insieme organizzato di dati utilizzati per il supporto allo svolgimento di attività (di un ente, azienda, ufficio, persona).

Sistema informatico:

Un sistema informatico è una porzione automatizzata del sistema informativo, la parte del sistema informativo che gestisce informazioni con tecnologia informatica. Un sistema informatico garantisce che i dati siano conservati in modo permanente sui dispositivi di memorizzazione, permette un rapido aggiornamento dei dati per riflettere rapidamente le loro variazioni e rende i dati accessibili alle interrogazioni degli utenti.

Basi di dati e sistemi di gestione di basi di dati:

L'approccio "convenzionale" alla gestione dei dati sfrutta la presenza di archivi o file per memorizzare e ricercare dati, ma fornisce solo semplici meccanismi di accesso e condivisione.

Un sistema di gestione di basi di dati (**Data Base Management System, DBMS**) è un sistema software in grado di gestire collezioni di dati che siano grandi, condivise e persistenti, assicurando la loro affidabilità e privacy. Un DBMS deve essere efficiente ed efficace. Una base di dati è una collezione di dati gestita da un DBMS.

Caratteristiche delle basi di dati:

- **Grandi:** le basi di dati sono di dimensioni molto maggiori rispetto alla memoria centrale dei sistemi di calcolo utilizzati.
- **Persistenti:** le basi di dati hanno un tempo di vita indipendente dalle singole esecuzioni dei programmi che le utilizzano.
- **Condivise:** diverse applicazioni e utenti devono poter accedere a dati comuni. È importante notare che in questo modo si riduce la ridondanza dei dati, poiché si evitano le ripetizioni, e si riduce anche la possibilità di inconsistenze.
- **Affidabili:** capacità del sistema di conservare intatto il contenuto della base di dati.
- **Private:** ciascun utente viene abilitato a svolgere determinate azioni sui dati in base a meccanismi di autorizzazione.
- **Efficienti:** sono capaci di svolgere le operazioni utilizzando un insieme di risorse che sia accettabile per gli utenti.
- **Efficaci:** sono capaci di rendere produttive le attività dei loro utenti.

Modelli di dati:

Un modello di dati è un insieme di costrutti utilizzati per organizzare i dati di interesse e descriverne la struttura in modo che essa risulti comprensibile a un elaboratore. Ogni modello di dati fornisce

meccanismi di strutturazione che permettono di definire nuovi tipi sulla base di tipi predefiniti e costruttori di tipo.

Il modello relazionale dei dati, attualmente il più diffuso, permette di definire tipi per mezzo del costruttore relazione, che consente di organizzare i dati in insiemi di record a struttura fissa.

Organizzazione dei dati in una base di dati:

Schemi e istanze: Nelle basi di dati esiste una parte detta “schema della base di dati”, costituita dalle caratteristiche dei dati, e una parte detta “istanza”, costituita dai valori effettivi.

- Lo schema costituisce l’aspetto intensionale, ovvero la descrizione "astratta" delle proprietà, ed è invariante nel tempo.
- L'istanza costituisce l’aspetto estensionale "concreto", che varia nel tempo al variare della situazione di ciò che stiamo descrivendo.

Esistono due tipi di modelli:

1. Modelli logici: un modello logico è adottato nei DBMS esistenti per l’organizzazione dei dati.
2. Modelli concettuali: permettono di rappresentare i dati in modo indipendente da ogni sistema, cercano di descrivere i concetti del mondo reale e sono utilizzati nelle fasi preliminari della progettazione.

Architettura dei DBMS:

Esiste un’architettura standardizzata per DBMS a tre livelli, detti rispettivamente esterno, logico e interno:

- **Schema logico:** descrizione dell’intera base di dati per mezzo del modello logico adottato dal DBMS.
- **Schema interno:** costituisce la rappresentazione dello schema logico per mezzo di strutture fisiche di memorizzazione.
- **Schema esterno:** costituisce la descrizione di una porzione della base di dati di interesse per mezzo del modello logico.

Modello relazionale:

Un modello dei dati rappresenta un insieme di concetti fondamentali per organizzare e descrivere la struttura dei dati. Al centro di ogni modello vi sono i meccanismi di strutturazione, che svolgono un ruolo analogo ai costruttori di tipo, consentendo la definizione di nuovi tipi basati su quelli predefiniti. Inoltre, ogni modello dei dati offre una serie di costruttori che permettono di creare nuovi tipi a partire da tipi elementari predefiniti. Tra i vari modelli dei dati, quello relazionale è il più ampiamente diffuso, caratterizzato dalla strutturazione dei dati in tabelle e dalla definizione di relazioni tra di esse.

Il costruttore di relazioni permette di definire tipi che consentono l'organizzazione dei dati in insiemi di record con struttura fissa.

- **Righe:** rappresentano specifici record.
- **Colonne:** corrispondono ai campi dei record.

Relazioni nel modello relazionale:

Ad ogni dominio è assegnato un nome, noto come attributo, che ne indica la sua funzione o ruolo. Gli attributi possono essere impiegati come intestazioni e la struttura non è basata sulla posizione dei dati.

Tabelle e relazioni:

Una tabella viene considerata una relazione quando i valori in ciascuna colonna sono omogenei e le righe sono distinte l'una dall'altra. Inoltre, le intestazioni delle colonne devono essere diverse tra loro. In una tabella che rappresenta una relazione, l'ordine delle righe e delle colonne non ha rilevanza; ciò significa che non importa in quale sequenza le righe o le colonne sono organizzate, poiché la struttura della relazione è definita dai valori all'interno della tabella, non dal loro posizionamento.

Struttura basata sui valori:

La struttura dei dati è indipendente dalle loro rappresentazioni fisiche, che potrebbero variare dinamicamente, inclusi gli usi di puntatori di alto livello. Si descrive solo ciò che è rilevante per l'applicazione, rendendo i dati più facilmente trasferibili da un sistema all'altro. Inoltre, per accedere ai dati, non è necessario conoscere il modo in cui sono memorizzati fisicamente.

Schema e istanza:

In una base di dati, si distinguono due aspetti fondamentali: lo schema e l'istanza.

- Lo schema, che rimane sostanzialmente invariante nel tempo, descrive la struttura di base dei dati, come ad esempio le intestazioni delle tabelle.
- L'istanza rappresenta i valori attuali dei dati, i quali possono variare rapidamente nel tempo, costituendo il "corpo" di ciascuna tabella o entità.

Definizioni del modello relazionale:

- Chiave: la chiave è un insieme di attributi che identificano univocamente le tuple di una relazione.
- Superchiave: una superchiave è un insieme di uno o più attributi di una relazione (tabella) tale per cui, all'interno di ogni tupla (riga) della relazione, i valori di tali attributi sono univoci. Quindi una superchiave è un insieme minimo di attributi che può essere utilizzato per identificare univocamente ogni tupla all'interno della relazione.
- Superchiave minimale: la superchiave minimale è un insieme minimo di attributi che è ancora in grado di identificare univocamente ogni tupla all'interno della relazione. Se si rimuovesse anche un solo attributo dalla superchiave minimale, questa non sarebbe più in grado di garantire l'unicità delle tuple.

Informazione incompleta nel modello relazionale:

Per indicare un'informazione incompleta nel modello relazionale si utilizza una tecnica semplice ma efficace che comprende l'uso del valore nullo, il quale indica l'assenza di un valore nel dominio specifico e non è considerato un valore valido nel dominio stesso.

Vincoli di integrità:

I vincoli di integrità sono regole o condizioni che devono essere rispettate affinché i dati rimangano coerenti, corretti e affidabili. Questi vincoli assicurano che le informazioni contenute nella base dati restino corrette e significative.

Tipi di vincoli di integrità:

- **Vincoli intrarelazionali:**
 - Vincoli di dominio.
 - Vincoli di tupla:

- Vincoli di chiave
- **Vincoli interrelazionali (coinvolgono più relazioni):**
 - Vincolo referenziale.

Vincoli di tupla:

Un vincolo di tupla è una condizione applicata a ogni tupla di una tabella (relazione). In genere la condizione è un'espressione booleana o aritmetica che verifica i valori degli attributi dentro la tupla. Questo vincolo limita il range dei valori nel dominio di un attributo.

Esempio: in un database possono essere inserite solo le persone con cittadinanza italiana minorenni. Ci sono due vincoli di tupla: L'attributo cittadinanza deve essere uguale a "Italia" e l'attributo età deve essere minore di 18 e maggiore di 0.

Vincoli di chiave:

I vincoli di chiave sono gli attributi scelti come chiave della tabella (relazione). La chiave è l'attributo o l'insieme di attributi che permette di accedere univocamente a ogni tupla della tabella. Esistono diversi tipi di chiavi, **la più importante è la chiave primaria che non ha valori duplicati, né valori nulli.** Una tabella può avere una sola chiave primaria.

NOTA BENE!

Ogni relazione ha almeno una chiave, che può essere l'intera superchiave (nel caso in cui tutti gli attributi siano necessari per identificare univocamente una tupla) o un sottoinsieme di essa (nel caso in cui solo alcuni attributi siano necessari per identificare univocamente una tupla).

Vincolo di integrità referenziale:

Il vincolo di integrità referenziale è una regola del modello relazionale che garantisce l'integrità dei dati quando si hanno relazioni associate tra loro attraverso la chiave esterna. Queste regole servono per rendere valide le associazioni tra le tabelle e per eliminare gli errori di inserimento, cancellazione o modifica di dati collegati tra loro.

Esempio: Ad esempio, considera due tabelle: una tabella Studenti e una tabella Corsi. Se un record nella tabella Corsi ha un campo id_studente che fa riferimento all'id di un record nella tabella Studenti, il vincolo di integrità referenziale assicura che non possiamo eliminare lo studente dalla tabella Studenti senza prima aver eliminato o modificato tutti i corsi associati a quell'id_studente nella tabella Corsi. Questo previene la presenza di "dati orfani" (ovvero, corsi che fanno riferimento a uno studente che non esiste più).

Riassunto delle definizioni:

TERMINI INFORMALI	TERMINI FORMALI
TABELLA	RELAZIONE
COLONNA	ATTRIBUTO/DOMINIO
RIGA	TUPLA
VALORI NELLA COLONNA	DOMINIO
DEFINIZIONE DELLA TABELLA	SCHEMA DELLA RELAZIONE
TABELLA POPOLATA	ISTANZA DELLA RELAZIONE

- **Superchiave:** una superchiave è un insieme di attributi che rendono univoco ogni record.

- **Superchiave non minimale:** è una superchiave dalla quale posso togliere almeno un attributo ed ottenere comunque una superchiave.

Algebra relazionale:

Una relazione è un insieme quindi ha le seguenti caratteristiche:

1. Non c'è ordinamento tra le diverse tuple.
2. Le tuple sono distinte (non ce ne possono essere due uguali).
3. Ciascuna tupla è al suo interno ordinata: l'i-esimo valore proviene dall'i-esimo dominio.

Una tabella è una relazione SOLO se non ci sono righe uguali.

Operatori insiemistici:

- Unione;
- Intersezione;
- Differenza insiemistica.

È possibile applicare unione, intersezione, differenza solo a relazioni definite sugli stessi attributi. Per poter rilasciare questo vincolo abbiamo bisogno di potere uniformare i nomi degli attributi.

Operatore di ridenominazione:

$$\rho_{y \leftarrow x}(r)$$

Si tratta di un operatore monadico, cioè con un solo argomento. Agisce cambiando il nome dell'attributo x della relazione r in y. Modifica lo schema lasciando inalterata l'istanza di r. Se è necessaria la modifica di più attributi è importante guardare l'ordine degli stessi.

Operatore di proiezione:

$$\Pi_{\text{listaAttributi}}(r)$$

È un operatore monadico che produce una relazione definita su una parte degli attributi della relazione r, quelli specificati nella ListaAttributi che contiene ennuple a cui contribuiscono tutte le n-uple di r.

Operatore selezione:

$$\sigma_{\text{condizione}}(r)$$

La selezione genera una relazione R avente lo stesso schema di A e contenente tutte le tuple della relazione A per cui è vera la condizione. La condizione è un'estensione booleana di espressioni di confronto tra attributo o tra attributi e costanti.

Condizione (o formula proposizionale):

Data una relazione r, la condizione è una espressione booleana ottenuta combinando con i connettivi OR ($\vee$), AND ($\wedge$), e NOT ($\neg$) condizioni atomiche.

Operatore join:

L'operatore join produce il sottoinsieme del prodotto cartesiano di due relazioni in cui il valore di determinati attributi coincide.

Esistono diverse tipologie di join:

- Join naturale.
- Prodotto cartesiano.
- Theta join.

- Join esterno (destro, sinistro, completo).

Join naturale:

Produce una relazione definita sull'unione x_1, x_2 degli attributi degli operandi composta da n-ple t che rispettano la condizione:

$$\{t \text{ su } X_1 \cup X_2 | t[x_1] \in r_1 \text{ e } t[x_2] \in r_2\}$$

Se due relazioni sono definite sugli stessi attributi il join naturale equivale all'intersezione delle due relazioni.

Un join naturale su relazioni che non hanno attributi in comune produce il prodotto cartesiano delle relazioni, perché non essendoci un criterio di combinazione tutte le coppie di ennuple sono combinabili tra loro, e quindi esso contiene sempre un numero di ennuple uguale al prodotto delle cardinalità degli operandi.

Theta join:

Il theta-join di due relazioni A e B genera una relazione R avente come schema l'unione degli schemi di A e B e contenente tutte le coppie costituite da una tupla di A e una tupla di B per cui è vera una certa condizione.

- La condizione del theta join è tipicamente una congiunzione (AND) di espressioni di confronto $A \theta B$ dove θ è uno degli operatori di confronto.
- Se l'operatore utilizzato nella condizione è l'uguaglianza ($\theta: =$) allora si parla di equi-join.

Tipologie di join esterno:

- Sinistro (Left Join): mantiene tutte le ennuple del primo operando, estendendole con valori nulli, se necessario.
- Destro (Right join): mantiene tutte le ennuple del secondo operando.
- Completo (full join): mantiene tutte le ennuple di entrambi gli operandi.

Self-join:

Si effettua il join di una relazione con una copia sé stessa in modo tale da:

- Confrontare tuple di una relazione tra loro
- Trovare elementi duplicati.
- Quando una tupla fa riferimento a dati nella stessa tabella.

Valori nulli:

Per riferirsi ai valori nulli nelle basi di dati esistono apposite condizioni:

- IS NULL;
- IS NOT NULL.

Effetti dei valori nulli sugli operatori AR:

- Unione, intersezione e differenza continuano a comportarsi usualmente, quindi due tuple sono uguali anche se ci sono dei NULL.
- Il join naturale non combina due tuple se una delle due tuple o entrambe hanno valore NULL su un attributo in comune (e valori uguali sugli eventuali altri attributi comuni).

- Per la selezione il problema è stabilire se, in presenza di NULL, un predicato è vero o meno per una data tupla => si usa una logica a 3 valori.

Viste:

Le viste sono relazioni virtuali, non sono effettivamente memorizzate nella base di dati. L'utilizzo delle viste permette di evitare ridondanze. Un'interrogazione su una vista viene eseguita "ricalcolando" una vista.

Progettazione logica:

L'obiettivo della progettazione logica è la traduzione dello schema concettuale in uno schema logico che rappresenti gli stessi dati in maniera corretta ed efficiente.

Ristrutturazione dello schema E-R:

Nella ristrutturazione dello schema E/R vengono eliminati tutti i costrutti che non possono essere direttamente rappresentati nel modello logico target attraverso:

- Eliminazione degli attributi multivalore;
- Eliminazione delle generalizzazioni;
- Partizionamento/accorpamento di entità e associazioni;
- Scelta degli identificatori primari;
- Analisi delle ridondanze.

Per ottimizzare il risultato è necessario analizzare le prestazioni, tuttavia quest'ultime non sono valutabili con precisione su uno schema concettuale ma dipendono dalle caratteristiche del DBMS.

Per valutare le prestazioni di un sistema di gestione dei dati, è fondamentale considerare due indicatori principali:

- Il tempo di esecuzione delle operazioni di interesse;
- Lo spazio di memoria necessario per memorizzare i dati rilevanti.

Il tempo di esecuzione dipende dal numero medio di istanze di entità e relazioni accedute durante le operazioni, mentre lo spazio di memoria è determinato dal volume complessivo dei dati, che include il numero previsto di istanze e la dimensione di ciascun attributo.

Inoltre, bisogna tenere conto delle caratteristiche delle operazioni, come il tipo (interattiva o batch), la frequenza delle esecuzioni e i dati coinvolti.

Schema di operazione:

Lo schema di operazione descrive i dati coinvolti in un'operazione, corrisponde al frammento dello schema ER interessato all'operazione sul quale viene disegnato il cammino logico per accedere alle informazioni di interesse.

Analisi delle ridondanze:

Una ridondanza in uno schema E-R è una informazione significativa ma derivabile da altre. Un vantaggio nell'eliminare una ridondanza consiste nella semplificazione delle interrogazioni; tuttavia, ci sarebbe una maggiore occupazione di spazio.

Le forme di ridondanza sono:

- Attributi derivabili;
- Associazioni derivabili.

Eliminazione delle gerarchie:

Il modello relazionale non può rappresentare direttamente le generalizzazioni a differenza di entità e relazioni che sono direttamente rappresentabili. Si eliminano pertanto le gerarchie sostituendole con entità e relazioni.

Ci sono tre possibilità:

- Accorpamento delle entità figlie nell'entità padre;
- Accorpamento dell'entità padre nelle entità figlie;
- Sostituzione della generalizzazione con una relazione.

Accorpamento delle entità figlie nell'entità padre:

Vengono eliminate le entità figlie, i loro attributi sono aggiunti all'entità padre. Si aggiunge un attributo per distinguere il tipo. Gli attributi che provengono da una figlia possono essere nulli.

- **Utilizzo:** conviene se gli accessi al padre e alle figlie sono contestuali.

Accorpamento dell'entità padre nelle entità figlie:

L'entità padre viene eliminata, gli attributi, le associazioni e l'identificatore del padre sono aggiunti alle figlie e ogni associazione definita sul padre genera una associazione distinta per ogni figlia.

- **Utilizzo:** conviene se gli accessi alle figlie sono distinti e solo se la generalizzazione è totale.

Sostituzione della generalizzazione con una relazione

Si introduce una relazione uno-a-uno fra l'entità padre e ciascuna entità figlia, occorre inserire il vincolo che ogni istanza dell'entità padre può partecipare solo ad una relazione di legame con le entità figlie. Se la generalizzazione è totale ogni istanza dell'entità padre partecipa necessariamente ad una (sola) delle relazioni di legame con le figlie.

- **Utilizzo:** conviene se gli accessi alle entità figlie sono separati dagli accessi al padre.

Eliminazione degli attributi multivalore:

Un attributo multivalore diventa un'entità collegata con una relazione (uno a molti).

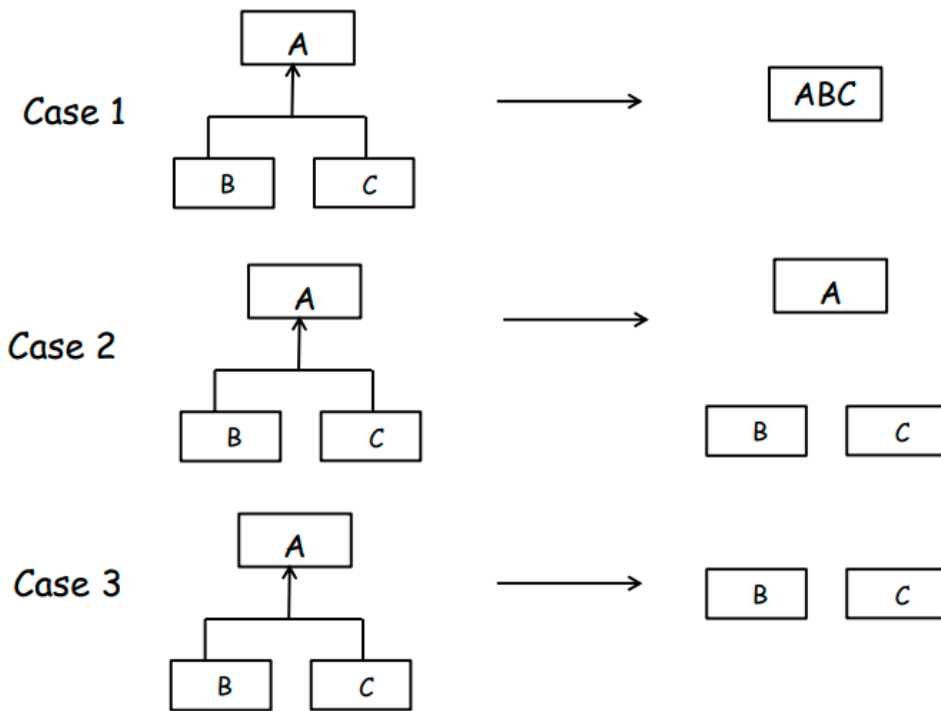
Scelta degli identificatori primari:

I criteri per la scelta degli identificatori primari sono i seguenti:

- Assenza di opzionalità (no valori nulli)
- Semplicità (preferenza agli identificatori interni, dimensioni ridotte)
- Utilizzo nelle operazioni più frequenti o importanti.

Se nessuno degli identificatori soddisfa i requisiti, si introducono nuovi attributi (codici) contenenti valori speciali generati appositamente per questo scopo.

Ristrutturazione delle generalizzazioni:

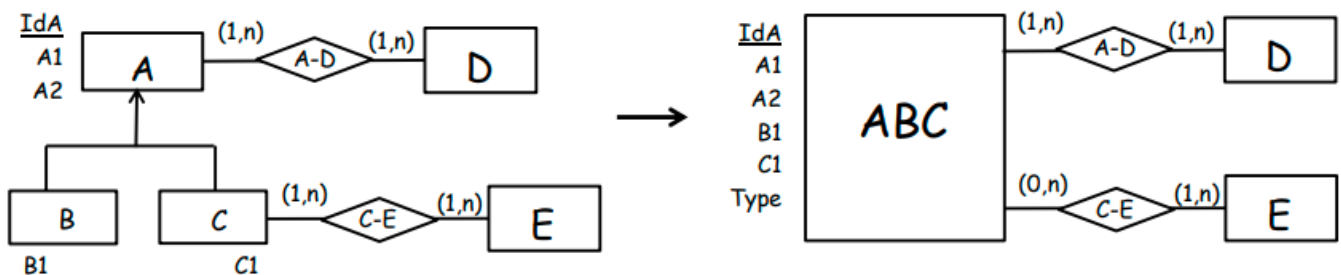


La scelta tra i tre casi dipende dalle caratteristiche del carico della query e implica scelte di ottimizzazione, i tre casi sono:

1. **Accorpamento delle figlie nel padre:** si verifica quando le query visitano tutte e tre le entità (o n nel caso generale) insieme.
2. **Accorpamento del padre nelle figlie:** si verifica quando le query visitano le due entità figlie nella generalizzazione e non l'entità padre. Per poter effettuare questa operazione la generalizzazione deve essere totale.
3. **Sostituzione della generalizzazione con le relazioni:** si verifica quando le query visitano separatamente le tre entità, cioè quando gli accessi dell'entità padre sono separati da quelli dell'entità figlie.

La trasformazione deve essere eseguita in modo da mantenere inalterato il contenuto informativo dello schema. È importante associare con attenzione le proprietà (attributi e relazioni) della nuova entità o delle nuove entità alle entità della generalizzazione nello schema originale.

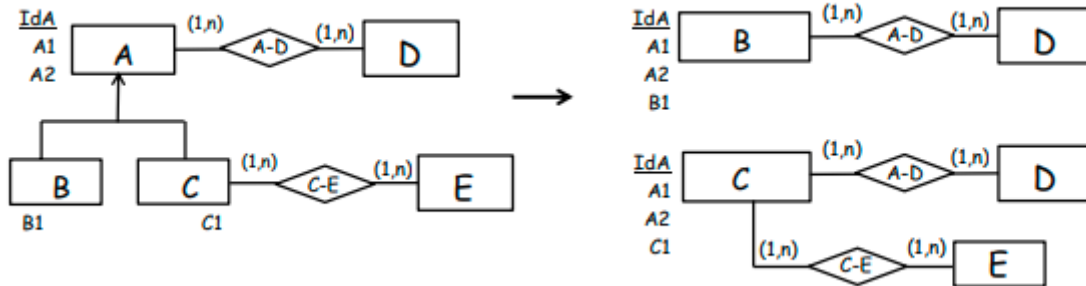
CASO 1:



Vengono assegnati all'unica entità ABC risultante dalla trasformazione tutti gli attributi e le relazioni precedentemente assegnate ad A, B e C. Inoltre, viene aggiunto un nuovo attributo Tipo all'entità ABC. Infine, viene modificata la cardinalità minima di ABC nella relazione C-E, poiché le istanze di

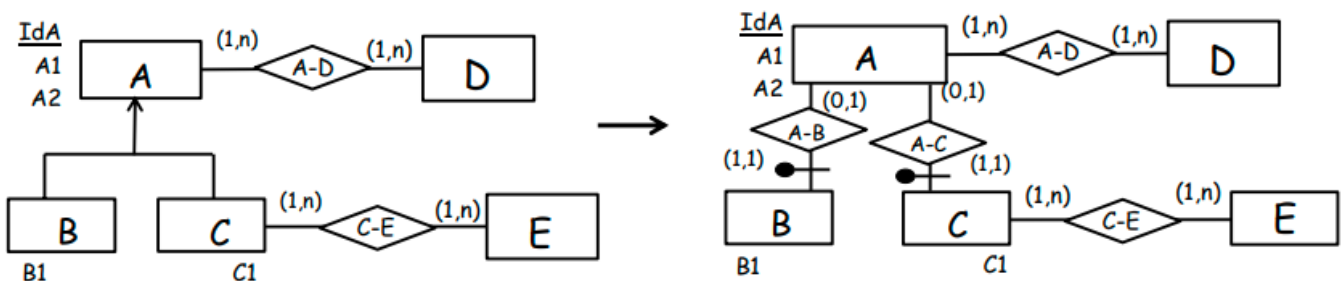
C sono un sottoinsieme delle istanze di A e non possiamo essere sicuri che tutte le istanze di A siano collegate con istanze di E.

CASO 2:



Vengono assegnate le proprietà di A sia a C che a B e vengono mantenute le proprietà che B e C avevano in precedenza.

CASO 3:

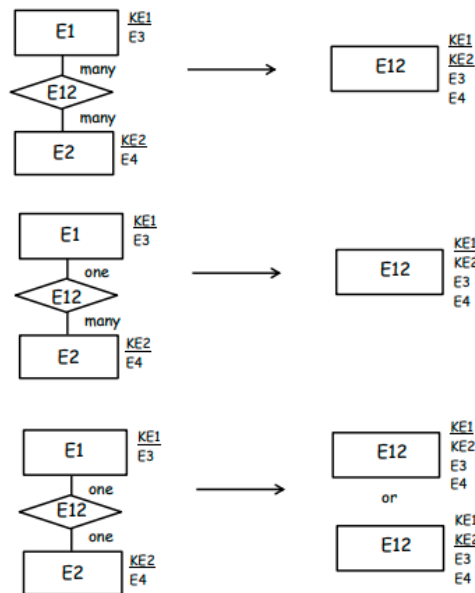


Vengono collegate le entità B e C con A attraverso due relazioni distinte per rappresentare le stesse istanze nelle coppie A-B e A-C. Inoltre, poiché le istanze di B e C coincidono con un sottoinsieme delle istanze di A, aggiungiamo a B e C un identificatore esterno. Le due cardinalità minime di A nelle relazioni A-B e A-C sono entrambe 0, poiché alcune delle istanze di A, sono comuni a B e altre sono comuni a C.

OTTIMIZZAZIONE FUSIONE:

La fusione di due entità (e della relazione correlata) viene eseguita quando nel carico della query le query visitano insieme le due entità e la relazione. L'identificatore e gli attributi dell'entità risultante dipendono dal tipo di relazione, ovvero:

1. Molti a molti;
2. Molti a uno;
3. Uno a uno.



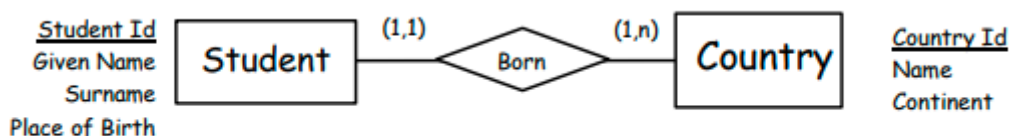
1. **Molti a molti:** Quando la relazione è molti a molti, l'identificatore dell'entità E12 è l'unione degli identificatori delle entità da unire. La ragione di questa regola è che in questo caso le istanze dell'entità corrispondono alle istanze molti a molti della relazione e, quindi, per essere identificate, dobbiamo conoscere gli identificatori delle istanze di entrambe le entità E1 ed E2.
2. **Uno a molti:** Quando la relazione è uno a molti, l'identificatore dell'entità E12 è l'identificatore dell'entità sul lato uno, poiché l'istanza corrispondente identifica anche l'istanza unica dell'entità ad essa correlata sul lato molti.
3. **Uno a uno:** Quando la relazione è uno a uno, possiamo scegliere l'identificatore della prima entità o l'identificatore della seconda entità.

Passaggi di traduzione:

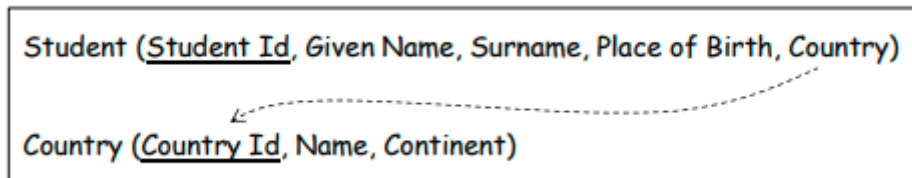
- Relazione molti a molti
- Relazione ternaria molti a molti
- Relazione ricorsiva molti a molti
- Relazione uno a molti
- Entità con identificatore esterno
- Relazione uno a uno; dovremo discutere tre casi:
 - (1,1) a (1,1)
 - (0,1) a (1,1)
 - (0,1) a (0,1)

Relazione uno a molti:

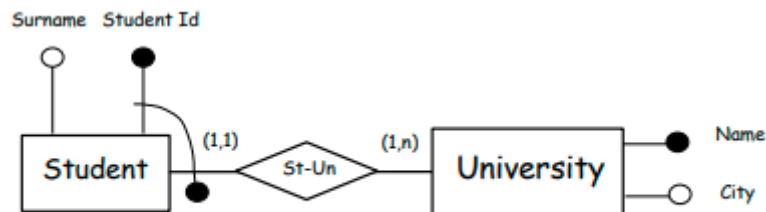
Una relazione uno a molti è un particolare tipo di relazione molti, in questo caso per ogni istanza di studente c'è un'unica corrispondente istanza di paese nella relazione nato. Pertanto, si può evitare di creare tre tabelle distinte inserendo la chiave countryId tra gli attributi della tabella studente.



Example of one to many relationship

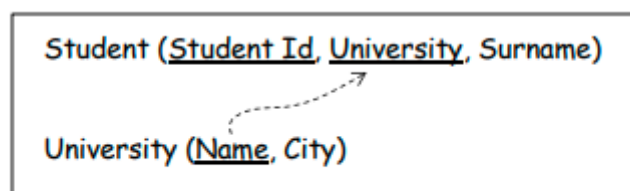


Relazione con identificatore esterno:



The example for translation of an entity with external identifier

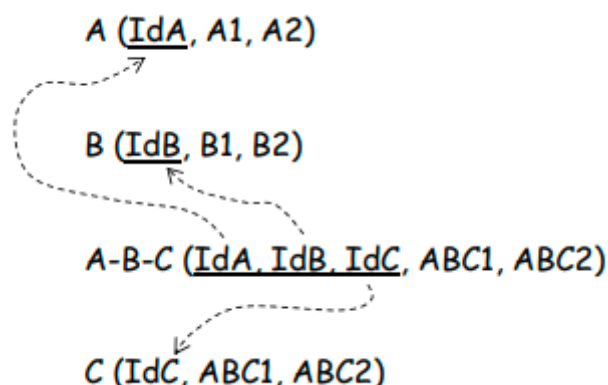
L'identificatore esterno comporta che la chiave della tabella studente venga estesa con l'identificatore (chiave primaria) della seconda entità coinvolta nella relazione. Pertanto, la coppia di tabelle risultante è la seguente:



Resulting relational schema

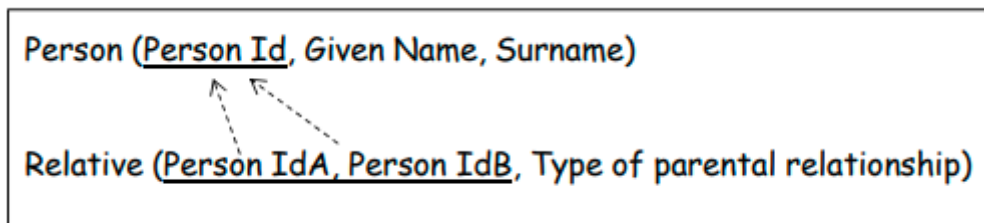
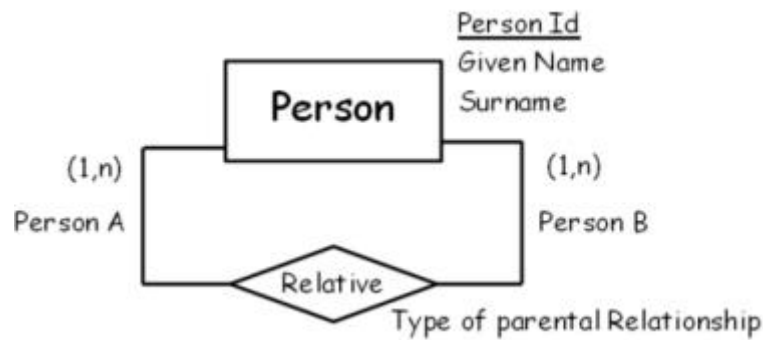
Relazione ternaria molti a molti:

La relazione ternaria molti a molti si comporta come quella semplice, l'unica modifica è che ora le istanze della relazione sono triple costituite da istanze di tre entità; quindi, abbiamo una chiave composta da tre attributi per la relazione corrispondente a A-B-C.



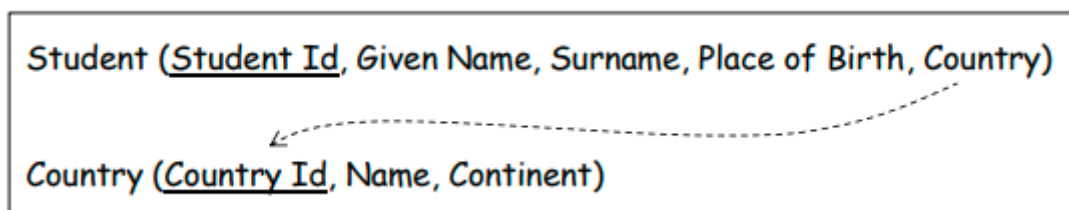
Relazione molti a molti ricorsiva:

La relazione ricorsiva molti a molti è un'estensione dell'a relazione molti a molti; tuttavia, in questo le due entità coinvolte nella relazione coincidono pertanto la chiave primaria è costituita da due occorrenze ricavate dalla chiave primaria della tabella denominate in modi differenti.

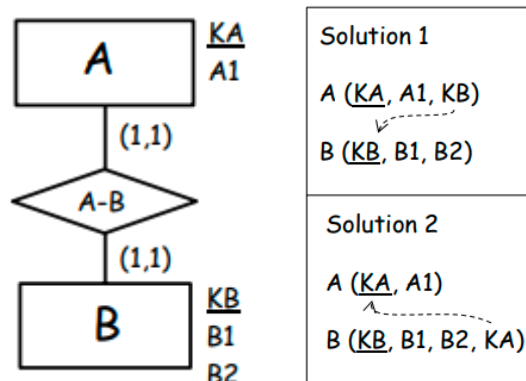


Relazione uno a molti:

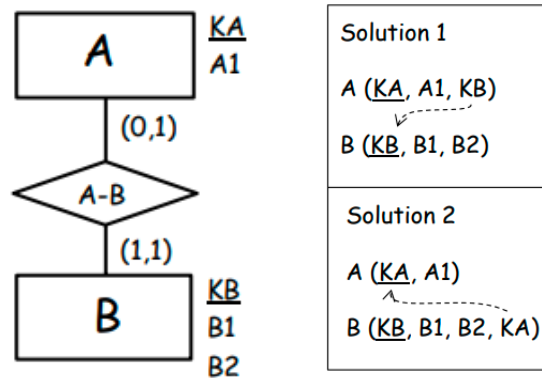
Anche una relazione uno a molti è un caso particolare di una relazione molti a molti, quindi potremmo tradurre le due entità e la relazione con tre tabelle. Tuttavia, dobbiamo osservare che, a causa della relazione uno a molti, per ogni istanza di Studente abbiamo un'istanza corrispondente unica di Paese nella relazione nascita. Di conseguenza, possiamo risparmiare una tabella includendo Course Id tra gli attributi della tabella Studente. Otteniamo quindi il seguente schema relazionale.



Relazione uno a uno: caso (1,1) to (1,1)

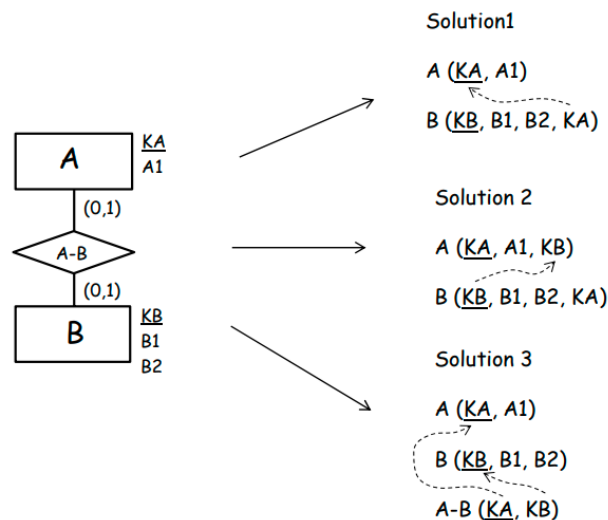


Relazione uno a uno: caso (0,1) to (1,1)



In questo caso la soluzione è preferibile in quanto B ha una cardinalità (1,1) pertanto non ci sono valori nulli dal momento che tutte le istanze di B sono connesse alle istanze di A e quindi la KA assumerà sempre valori non nulli.

Relazione uno a uno: caso (0,1) to (0,1)



Il motivo per cui si introduce il terzo caso è relativo al caso in cui entrambi le entità A e B hanno cardinalità uguale a 0. In questo caso sappiamo che solamente un numero limitato di istanze A e B sono collegate tramite la relazione A-B; pertanto, sarebbe preferibile introdurre una terza tabella corrispondente alla relazione A-B in cui si va a rappresentare solamente il paio di chiavi che si riferiscono alle istanze collegate di A e B.

Linguaggio SQL

SQL (Structured Query Language) è il linguaggio per la definizione e la manipolazione dei dati in database relazionali, adottato da tutti i principali DBMS.

SQL si compone di:

- **DDL (data Definition Language)**
 - Operazioni di definizione e modifica schema.
- **DML(Data Manipulation Language)**
 - Operazioni di interrogazione tramite SELECT.
 - Operazioni di aggiornamento (INSERT, UPDATE, DELETE).

Istruzioni principali in SQL:

Operazioni di definizione schema e modifica (DDL):

- Create: definisce database, tabelle, domini, viste, vincoli e autorizzazioni.
- Alter: modifica attributi e vincoli.
- Drop: elimina database e tabelle.

Operazioni di interrogazione (DML):

- Select: formula query.

Operazioni di aggiornamento (DML):

- Insert: inserisce nuove tuple nelle tabelle.
- Delete: elimina tuple nelle tabelle.
- Update: modifica tuple.

SQL è un linguaggio dichiarativo, pertanto non possiamo scegliere l'ordine con cui avvengono le operazioni.

Istruzione SELECT:

- Clausola **SELECT** tramite cui scelgo le colonne.
- Clausola **FROM**: tabella/e da cui voglio estrarre le informazioni. Se si indicano più tabelle viene considerato il loro prodotto cartesiano, a prescindere dagli schemi delle tabelle.
- Clausola **WHERE** tramite cui scelgo le righe.

SELECT ListaAttributi

FROM ListaTabelle

[**WHERE** Condizione]

Per ottenere tutti gli attributi di una tabella si utilizza la clausola *.

Condizione Like:

Seleziona stringhe di caratteri, utilizza due caratteri speciali da inserire dentro le stringhe.

- `_` indica un singolo carattere arbitrario.
- `%` indica una stringa con un numero arbitrario di caratteri (da 0 a n caratteri, potenzialmente 0).

Esempio:

```
SELECT *  
FROM docente  
WHERE nome like 'A_d%'
```

Seleziona i docenti che:

Hanno un nome che inizia per 'A' e ha una 'd' come terza lettera.

Creare tabelle in SQL:

Tramite l'istruzione create è possibile creare delle tabelle in SQL, l'istruzione CREATE TABLE definisce uno schema di relazione e ne crea un'istanza vuota, cioè senza ennuple.

```
CREATE TABLE TableName  
(AttributeName Domain [DefaultValue] [Constraints]  
{AttributeName Domain [DefaultValue] [Constraints]}  
[OtherConstraints])
```

Domini elementary predefiniti

L'istruzione CREATE DOMAIN definisce un dominio (elementare), utilizzabile in definizioni di relazioni, anche con vincoli e valori di default.

Vincoli di integrità:

- Intra-relazionali: proprietà sempre valida all'interno di una relazione.
- Inter-relazionali: proprietà sempre valida relativa a più relazioni.

Vincoli intrarelazionali:

- NOT NULL: indica che il valore nullo non è ammesso per il dominio a cui è associato il vincolo.
- UNIQUE: Impone che i valori dell'attributo o dell'insieme di attributi siano una chiave, cioè che tuple diverse abbiano valori diversi.
- PRIMARY KEY: serve per specificare la chiave primaria, può essere utilizzata una sola volta in una tabella.

Vincoli interrelazionali

- FOREIGN KEY e REFERENCES vincoli di integrità referenziale: creano un legame tra i valori di n attributi della tabella corrente e i valori di n attributi di una seconda tabella (detta tabella master) di cui costituiscono una chiave.
- CHECK.

Vincoli di propagazione:

I vincoli di propagazione consentono di definire la modalità con cui propagare gli aggiornamenti riguardanti valori oggetto di un vincolo di integrità referenziale verso la tabella in cui il vincolo è definito.

- CASCADE: l'operazione di modifica/cancellazione viene propagata alla tabella interna.
- SET NULL: viene impostato il valore a NULL.
- SET DEFAULT: valore impostato al valore di default della colonna.

- **NO ACTION:** se l'operazione viola l'integrità, viene negata.

Costrutto Join SQL:

Il costrutto join in SQL è utilizzato per unire i dati o le righe di due o più tabelle in base ad una chiave che hanno in comune.

Esistono diversi tipi di costrutti Join in SQL:

- Inner join.
- Left join.
- Right join.
- Full join.

Inner Join

La keyword inner join restituisce le righe quando c'è una corrispondenza in entrambe le tabelle.

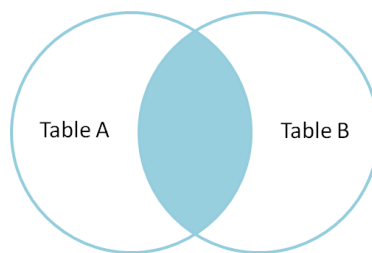
```
SELECT table1.column1,table1.column2,table2.column1,....
```

```
FROM table1
```

```
INNER JOIN table2 ON table1.matching_column = table2.matching_column;
```

table1: First table.

table2: Second table matching_column: Column common to both the tables.



Left join:

La keyword left join restituisce tutte le righe dalla tabella di sinistra e le righe corrispondenti dalla tabella di destra. Se non c'è corrispondenza, il risultato è NULL sulla parte destra

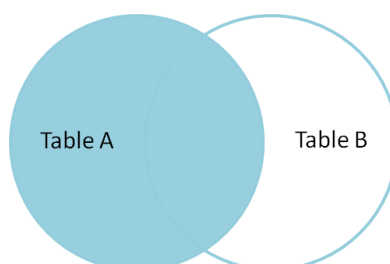
```
SELECT table1.column1,table1.column2,table2.column1,....
```

```
FROM table1
```

```
LEFT JOIN table2 ON table1.matching_column = table2.matching_column;
```

table1: First table.

table2: Second table matching_column: Column common to both the tables.



Right join:

La keyword right join restituisce tutte le righe dalla tabella di destra e le righe corrispondenti dalla tabella di sinistra. Se non c'è corrispondenza, il risultato è NULL sulla parte sinistra.

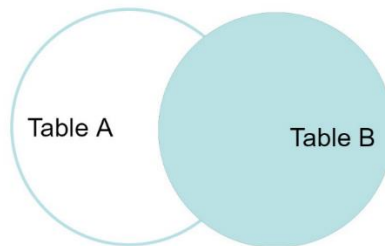
```
SELECT table1.column1,table1.column2,table2.column1,....
```

```
FROM table1
```

```
RIGHT JOIN table2 ON table1.matching_column = table2.matching_column;
```

table1: First table.

table2: Second table matching column: Column common to both the tables.



Full join:

La keyword full join restituisce le righe quando c'è una corrispondenza in una delle tabelle. Quindi, se c'è una riga in una tabella che non ha corrispondenze nell'altra tabella, il risultato JOIN è ancora restituito con NULL in ogni colonna della tabella che non ha una corrispondenza.

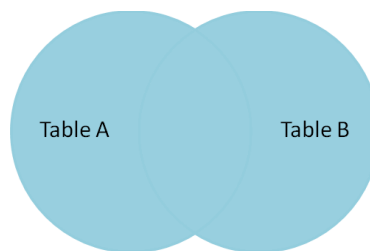
```
SELECT table1.column1,table1.column2,table2.column1,....
```

```
FROM table1
```

```
FULL JOIN table2 ON table1.matching_column = table2.matching_column;
```

table1: First table.

table2: Second table matching_column: Column common to both the tables.



Self join:

Un self join in SQL è un'operazione in cui una tabella viene unita a sé stessa. Questo tipo di join è utile quando i dati correlati sono memorizzati nella stessa tabella. Un self join può essere di qualsiasi tipo, come inner join o left join, e il risultato è una combinazione di righe dalla stessa tabella.

```
SELECT columns
```

```
FROM table AS alias1
```

```
JOIN table AS alias2 ON alias1.column = alias2.column;
```

Modifica delle tabelle in SQL:

Insert: viene utilizzato per inserire nuovi dati in una tabella.

```
INSERT INTO nome_tabella (colonna1, colonna2, colonna3)
VALUES (valore1, valore2, valore3);
```

Regole per il comando insert:

- Se qualche attributo non è specificato, è inserito valore NULL.
- Se è violato un vincolo di NOT NULL, l'insert è rifiutato.
- L'ordinamento degli attributi (se presente) e dei valori è significativo.
- Le due liste (Attributi, valori) devono avere lo stesso numero di elementi.
- Se la lista di attributi è omessa, si fa riferimento a tutti gli attributi della relazione, secondo l'ordine con cui sono definiti.
- Se la lista di attributi non contiene tutti gli attributi della relazione, per gli altri viene inserito un valore di default, o, in assenza di questo, il valore nullo (che deve essere permesso).

Delete: viene utilizzato per eliminare le righe esistenti in una tabella, se viene omessa la keyword WHERE, viene sottointeso il valore TRUE e quindi vengono cancellate tutte le ennuple.

```
DELETE FROM nome_tabella WHERE condizione;
```

Update: viene utilizzato per modificare i dati esistenti in una tabella.

```
UPDATE nome_tabella
SET colonna1 = valore1, colonna2 = valore2
WHERE condizione;
```

Vincolo di integrità Check:

Il vincolo CHECK viene utilizzato per limitare il valore che può essere inserito in una colonna o in un set di colonne.

```
CREATE TABLE persone (
    id INT PRIMARY KEY,
    nome VARCHAR(100),
    età INT CHECK (età > 0)
);
```

Asserzioni:

Le asserzioni rappresentano vincoli non associati a nessun attributo o tabella in particolare, ma appartengono direttamente allo schema della Base di Dati.

```
CREATE ASSERTION NomeAsserzione
check (Condizione);
```

Le asserzioni permettono di esprimere vincoli particolari, come vincoli su più tabelle o vincoli che richiedono una cardinalità minima\massima per una tabella.

Espressioni aritmetiche nelle target lists:

In SQL, le espressioni aritmetiche possono essere utilizzate nelle liste di attributi delle query, in modo tale da eseguire operazioni aritmetiche sui dati mentre vengono recuperati dal database. Gli operatori aritmetici sono: +, -, *, / e vengono usati seguendo le priorità matematiche.

Operatore BETWEEN:

L'operatore BETWEEN in SQL è utilizzato per selezionare valori all'interno di un determinato intervallo. I valori possono essere numeri, testo o date. L'operatore BETWEEN è inclusivo, il che significa che i valori di inizio e fine sono inclusi.

```
SELECT * FROM Prodotti WHERE Prezzo BETWEEN 10 AND 20;
```

Operatore ORDER BY:

L'operatore ORDER BY in SQL è utilizzato per ordinare i risultati di una query in un ordine specifico. Di default, ORDER BY ordina i risultati in ordine crescente. Per ordinare i risultati in ordine decrescente si può utilizzare la parola chiave DESC.

```
SELECT * FROM Clienti ORDER BY NomeCliente
```

Operatori aggregati:

Gli operatori aggregati in SQL sono funzioni specializzate che eseguono calcoli su gruppi di righe e restituiscono un unico risultato.

- **COUNT(column_name | *)**: Restituisce il numero di righe di una tabella. Può essere usato su tutti i tipi di dati.
- **SUM(column_name)**: Restituisce la somma dei valori di una colonna numerica. Può essere usato solo su dati numerici o intervalli di tempo.
- **AVG(column_name)**: Restituisce il valore medio di una colonna numerica. Può essere usato solo su dati numerici o intervalli di tempo.
- **MIN(column_name)**: Restituisce il valore minimo di una colonna selezionata. Può essere usato su qualsiasi dato ordinabile numerico o alfanumerico.
- **MAX(column_name)**: Restituisce il valore massimo di una colonna selezionata. Può essere usato su qualsiasi dato ordinabile numerico o alfanumerico.

Operatore GROUP BY:

L'operatore GROUP BY in SQL è utilizzato per raggruppare righe che hanno gli stessi valori in colonne specificate in un risultato aggregato. L'operatore group by è spesso utilizzato con funzioni aggregate come (COUNT(), MAX(), MIN(), SUM(), AVG()) per raggruppare il set di risultati in una o più colonne.

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);
```

Con l'operatore group by non è possibile utilizzare la clausola WHERE in quanto viene risolta prima dell'operatore stesso; pertanto, si utilizza una nuova keyword HAVING. La clausola HAVING permette di esprimere condizioni sui gruppi, applicate a ogni insieme di n-ple risultato della applicazione del GROUP BY.

```
SELECT column_name(s)
FROM table_name
ORDER BY column_name(s)
HAVING condition;
```

Subquery:

Una subquery è una query annidata all'interno di un'altra query SQL, può essere utilizzata per restituire dati che verranno utilizzati nella query principale come condizione per restringere ulteriormente i dati da recuperare.

Ci sono diversi tipi di subquery:

- **Subquery singola riga:** restituisce zero o una sola riga. Può essere utilizzata con operatori di confronto come =, >, <, >=, <=, <>.
- **Subquery multi-riga:** restituisce più di una riga. Può essere utilizzata con operatori IN, ANY, ALL e (NOT) EXISTS.

Operatori ALL e ANY:

Estendono i normali operatori di confronto,

- **ANY:** specifica che la condizione è vera se il confronto è vero rispetto ad almeno uno degli elementi di una lista.
- **ALL:** specifica che la condizione è vera se il confronto è vero rispetto a tutti gli elementi di una lista.

Operatori IN e NOT IN:

Gli operatori IN e NOT IN in SQL sono utilizzati per verificare se un valore esiste (o non esiste) in un elenco di valori:

- **IN:** ha valore VERO se il valore compare nell'insieme risultato della interrogazione.
- **NOT IN:** ha valore VERO se il valore non compare nell'insieme risultato della interrogazione.